JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ResourceBundle.Control

java.lang.Object
    java.util.ResourceBundle.Control

Enclosing class:

ResourceBundle

```
public static class ResourceBundle.Control
extends Object
```

`ResourceBundle.Control` defines a set of callback methods that are invoked by the `ResourceBundle.getBundle` factory methods during the bundle loading process. In other words, a `ResourceBundle.Control` collaborates with the factory methods for loading resource bundles. The default implementation of the callback methods provides the information necessary for the factory methods to perform the default behavior.

`ResourceBundle.Control` is designed for an application deployed in an unnamed module, for example to support resource bundles in non-standard formats or package localized resources in a non-traditional convention. `ResourceBundleProvider` is the replacement for `ResourceBundle.Control` when migrating to modules. `UnsupportedOperationException` will be thrown when a factory method that takes the `ResourceBundle.Control` parameter is called.

In addition to the callback methods, the `toBundleName` and `toResourceName` methods are defined primarily for convenience in implementing the callback methods. However, the `toBundleName` method could be overridden to provide different conventions in the organization and packaging of localized resources. The `toResourceName` method is `final` to avoid use of wrong resource and class name separators.

Two factory methods, `getControl(List)` and `getNoFallbackControl(List)`, provide `ResourceBundle.Control` instances that implement common variations of the default bundle loading process.

The formats returned by the `getFormats` method and candidate locales returned by the `getCandidateLocales` method must be consistent in all `ResourceBundle.getBundle` invocations for the same base bundle. Otherwise, the `ResourceBundle.getBundle` methods may return unintended bundles. For example, if only `"java.class"` is returned by the `getFormats` method for the first call to `ResourceBundle.getBundle` and only `"java.properties"` for the second call, then the second call will return the class-based one that has been cached during the first call.

A `ResourceBundle.Control` instance must be thread-safe if it's simultaneously used by multiple threads. `ResourceBundle.getBundle` does not synchronize to call the `ResourceBundle.Control` methods. The default implementations of the methods are thread-safe.

Applications can specify `ResourceBundle.Control` instances returned by the `getControl` factory methods or created from a subclass of `ResourceBundle.Control` to customize the bundle loading process. The following are examples of changing the default bundle loading process.

Example 1

The following code lets `ResourceBundle.getBundle` look up only properties-based resources.

```
import java.util.*;
import static java.util.ResourceBundle.Control.*;
...
ResourceBundle bundle =
  ResourceBundle.getBundle("MyResources", Locale.forLanguageTag("fr-CH"),
                           ResourceBundle.Control.getControl(FORMAT_PROPERTIES));
```

Given the resource bundles in the example in the `ResourceBundle.getBundle` description, this `ResourceBundle.getBundle` call loads `MyResources_fr_CH.properties` whose parent is `MyResources_fr.properties` whose parent is `MyResources.properties`. (`MyResources_fr_CH.properties` is not hidden, but `MyResources_fr_CH.class` is.)

Example 2

The following is an example of loading XML-based bundles using `Properties.loadFromXML`.

```
ResourceBundle rb = ResourceBundle.getBundle("Messages",
    new ResourceBundle.Control() {
        public List<String> getFormats(String baseName) {
            if (baseName == null)
                throw new NullPointerException();
            return Arrays.asList("xml");
        }
        public ResourceBundle newBundle(String baseName,
                                        Locale locale,
                                        String format,
                                        ClassLoader loader,
                                        boolean reload)
```

```
                        throws IllegalAccessException,
                               InstantiationException,
                               IOException {
            if (baseName == null || locale == null
                  || format == null || loader == null)
                throw new NullPointerException();
            ResourceBundle bundle = null;
            if (format.equals("xml")) {
                String bundleName = toBundleName(baseName, locale);
                String resourceName = toResourceName(bundleName, format);
                InputStream stream = null;
                if (reload) {
                    URL url = loader.getResource(resourceName);
                    if (url != null) {
                        URLConnection connection = url.openConnection();
                        if (connection != null) {
                            // Disable caches to get fresh data for
                            // reloading.
                            connection.setUseCaches(false);
                            stream = connection.getInputStream();
                        }
                    }
                } else {
                    stream = loader.getResourceAsStream(resourceName);
                }
                if (stream != null) {
                    BufferedInputStream bis = new BufferedInputStream(stream);
                    bundle = new XMLResourceBundle(bis);
                    bis.close();
                }
            }
            return bundle;
        }
    });

...

private static class XMLResourceBundle extends ResourceBundle {
    private Properties props;
    XMLResourceBundle(InputStream stream) throws IOException {
        props = new Properties();
        props.loadFromXML(stream);
    }
```

```
    protected Object handleGetObject(String key) {
        return props.getProperty(key);
    }
    public Enumeration<String> getKeys() {
        ...
    }
}
```

API Note:

`ResourceBundle.Control` is not supported in named modules. If the `ResourceBundle.getBundle` method with a `ResourceBundle.Control` is called in a named module, the method will throw an `UnsupportedOperationException`. Any service providers of `ResourceBundleControlProvider` are ignored in named modules.

Since:

1.6

See Also:

`ResourceBundleProvider`

Field Summary

Fields

Modifier and Type	Field	Description
`static final List<String>`	**`FORMAT_CLASS`**	The class-only format `List` containing `"java.class"`.
`static final List<String>`	**`FORMAT_DEFAULT`**	The default format `List`, which contains the strings `"java.class"` and `"java.properties"`, in this order.
`static final List<String>`	**`FORMAT_PROPERTIES`**	The properties-only format `List` containing `"java.properties"`.
`static final long`	**`TTL_DONT_CACHE`**	The time-to-live constant for not caching loaded resource bundle instances.
`static final long`	**`TTL_NO_EXPIRATION_CONTRO`**	

	The time-to-live constant for disabling the expiration control for loaded resource bundle instances in the cache.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`Control()`	Sole constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`List<Locale>`	**`getCandidateLocales`**`(String baseName, Locale locale)`	Returns a `List` of `Locales` as candidate locales for `baseName` and `locale`.
`static final ResourceBundle.Contr`	**`getControl`**`(List<String> formats)`	Returns a `ResourceBundle.Control` in which the `getFormats` method returns the specified `formats`.
`Locale`	**`getFallbackLocale`**`(String baseName, Locale locale)`	Returns a `Locale` to be used as a fallback locale for further resource bundle searches by the `ResourceBundle.getBundle` factory method.
`List<String>`	**`getFormats`**`(String baseName)`	Returns a `List` of `Strings` containing formats to be used to load resource bundles for the given `baseName`.
		Returns a `ResourceBundle.Control`

static final ResourceBundle.Contr	**getNoFallbackControl** (List<String> formats)	in which the getFormats method returns the specified formats and the getFallbackLocale method returns null.
long	**getTimeToLive** (String baseName, Locale locale)	Returns the time-to-live (TTL) value for resource bundles that are loaded under this ResourceBundle.Control.
boolean	**needsReload** (String baseName, Locale locale, String format, ClassLoader loader, ResourceBundle bundle, long loadTime)	Determines if the expired bundle in the cache needs to be reloaded based on the loading time given by loadTime or some other criteria.
ResourceBundle	**newBundle** (String baseName, Locale locale, String format, ClassLoader loader, boolean reload)	Instantiates a resource bundle for the given bundle name of the given format and locale, using the given class loader if necessary.
String	**toBundleName** (String baseName, Locale locale)	Converts the given baseName and locale to the bundle name.
final String	**toResourceName** (String bundleName, String suffix)	Converts the given bundleName to the form required by the ClassLoader.getResource method by replacing all occurrences of '.' in bundleName with '/' and appending a '.' and the given file suffix.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

FORMAT_DEFAULT

```
public static final List<String> FORMAT_DEFAULT
```

The default format `List`, which contains the strings `"java.class"` and `"java.properties"`, in this order. This `List` is unmodifiable.

See Also:

`getFormats(String)`

FORMAT_CLASS

```
public static final List<String> FORMAT_CLASS
```

The class-only format `List` containing `"java.class"`. This `List` is unmodifiable.

See Also:

`getFormats(String)`

FORMAT_PROPERTIES

```
public static final List<String> FORMAT_PROPERTIES
```

The properties-only format `List` containing `"java.properties"`. This `List` is unmodifiable.

See Also:

`getFormats(String)`

TTL_DONT_CACHE

```
public static final long TTL_DONT_CACHE
```

The time-to-live constant for not caching loaded resource bundle instances.

See Also:

`getTimeToLive(String, Locale)`,
Constant Field Values

TTL_NO_EXPIRATION_CONTROL

```
public static final long TTL_NO_EXPIRATION_CONTROL
```

The time-to-live constant for disabling the expiration control for loaded resource bundle instances in the cache.

See Also:

`getTimeToLive(String, Locale)`,
Constant Field Values

Constructor Details

Control

```
protected Control()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Details

getControl

```
public static final ResourceBundle.Control getControl
(List<String> formats)
```

Returns a `ResourceBundle.Control` in which the `getFormats` method returns the specified `formats`. The `formats` must be equal to one of `FORMAT_PROPERTIES`, `FORMAT_CLASS` or `FORMAT_DEFAULT`. `ResourceBundle.Control` instances returned by this method are singletons and thread-safe.

Specifying `FORMAT_DEFAULT` is equivalent to instantiating the `ResourceBundle.Control` class, except that this method returns a singleton.

Parameters:

`formats` - the formats to be returned by the `ResourceBundle.Control.getFormats` method

Returns:

a `ResourceBundle.Control` supporting the specified `formats`

Throws:

`NullPointerException` - if `formats` is `null`

`IllegalArgumentException` - if `formats` is unknown

getNoFallbackControl

```
public static final ResourceBundle.Control getNoFallbackControl
(List<String> formats)
```

Returns a `ResourceBundle.Control` in which the `getFormats` method returns the specified `formats` and the `getFallbackLocale` method returns `null`. The `formats` must be equal to one of `FORMAT_PROPERTIES`, `FORMAT_CLASS` or `FORMAT_DEFAULT`. `ResourceBundle.Control` instances returned by this method are singletons and thread-safe.

Parameters:

`formats` - the formats to be returned by the `ResourceBundle.Control.getFormats` method

Returns:

a `ResourceBundle.Control` supporting the specified `formats` with no fallback `Locale` support

Throws:

`NullPointerException` - if `formats` is `null`

`IllegalArgumentException` - if `formats` is unknown

getFormats

```
public List<String> getFormats(String baseName)
```

Returns a `List` of `String`s containing formats to be used to load resource bundles for the given `baseName`. The `ResourceBundle.getBundle` factory method tries to load resource bundles with formats in the order specified by the list. The list returned by this method must have at least one `String`. The predefined formats are `"java.class"` for class-based resource bundles and `"java.properties"` for properties-based ones. Strings starting with `"java."` are reserved for future extensions and must not be used by application-defined formats.

It is not a requirement to return an immutable (unmodifiable) `List`. However, the returned `List` must not be mutated after it has been returned by `getFormats`.

The default implementation returns `FORMAT_DEFAULT` so that the `ResourceBundle.getBundle` factory method looks up first class-based resource bundles, then properties-based ones.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

Returns:

a `List` of `Strings` containing formats for loading resource bundles.

Throws:

`NullPointerException` - if `baseName` is null

See Also:

`FORMAT_DEFAULT`, `FORMAT_CLASS`, `FORMAT_PROPERTIES`

getCandidateLocales

```
public List<Locale> getCandidateLocales(String baseName,
                                        Locale locale)
```

Returns a `List` of `Locales` as candidate locales for `baseName` and `locale`. This method is called by the `ResourceBundle.getBundle` factory method each time the factory method tries finding a resource bundle for a target `Locale`.

The sequence of the candidate locales also corresponds to the runtime resource lookup path (also known as the *parent chain*), if the corresponding resource bundles for the candidate locales exist and their parents are not defined by loaded resource bundles themselves. The last element of the list must be a root locale if it is desired to have the base bundle as the terminal of the parent chain.

If the given locale is equal to `Locale.ROOT` (the root locale), a `List` containing only the root `Locale` must be returned. In this case, the `ResourceBundle.getBundle` factory method loads only the base bundle as the resulting resource bundle.

It is not a requirement to return an immutable (unmodifiable) `List`. However, the returned `List` must not be mutated after it has been returned by `getCandidateLocales`.

The default implementation returns a `List` containing `Locales` using the rules described below. In the description below, *L*, *S*, *C* and *V* respectively represent non-empty language, script, country, and variant. For example, [*L*, *C*] represents a `Locale` that has non-empty values only for language and country. The form *L*("xx") represents the (non-empty) language value is "xx". For all cases, `Locales` whose final component values are empty strings are omitted.

1. For an input `Locale` with an empty script value, append candidate `Locales` by omitting the final component one by one as below:
    - [*L*, *C*, *V*]
    - [*L*, *C*]
    - [*L*]
    - `Locale.ROOT`

2. For an input `Locale` with a non-empty script value, append candidate `Locale`s by omitting the final component up to language, then append candidates generated from the `Locale` with country and variant restored:
    - [*L*, *S*, *C*, *V*]
    - [*L*, *S*, *C*]
    - [*L*, *S*]
    - [*L*, *C*, *V*]
    - [*L*, *C*]
    - [*L*]
    - `Locale.ROOT`
3. For an input `Locale` with a variant value consisting of multiple subtags separated by underscore, generate candidate `Locale`s by omitting the variant subtags one by one, then insert them after every occurrence of `Locale`s with the full variant value in the original list. For example, if the variant consists of two subtags *V1* and *V2*:
    - [*L*, *S*, *C*, *V1*, *V2*]
    - [*L*, *S*, *C*, *V1*]
    - [*L*, *S*, *C*]
    - [*L*, *S*]
    - [*L*, *C*, *V1*, *V2*]
    - [*L*, *C*, *V1*]
    - [*L*, *C*]
    - [*L*]
    - `Locale.ROOT`
4. Special cases for Chinese. When an input `Locale` has the language "zh" (Chinese) and an empty script value, either "Hans" (Simplified) or "Hant" (Traditional) might be supplied, depending on the country. When the country is "CN" (China) or "SG" (Singapore), "Hans" is supplied. When the country is "HK" (Hong Kong SAR China), "MO" (Macau SAR China), or "TW" (Taiwan), "Hant" is supplied. For all other countries or when the country is empty, no script is supplied. For example, for `Locale("zh", "CN")` , the candidate list will be:
    - [*L*("zh"), *S*("Hans"), *C*("CN")]
    - [*L*("zh"), *S*("Hans")]
    - [*L*("zh"), *C*("CN")]
    - [*L*("zh")]
    - `Locale.ROOT`

    For `Locale("zh", "TW")`, the candidate list will be:
    - [*L*("zh"), *S*("Hant"), *C*("TW")]
    - [*L*("zh"), *S*("Hant")]
    - [*L*("zh"), *C*("TW")]
    - [*L*("zh")]
    - `Locale.ROOT`

5. Special cases for Norwegian. Both `Locale("no", "NO", "NY")` and `Locale("nn", "NO")` represent Norwegian Nynorsk. When a locale's language is "nn", the standard candidate list is generated up to [*L*("nn")], and then the following candidates are added:
    - [*L*("no"), *C*("NO"), *V*("NY")]
    - [*L*("no"), *C*("NO")]
    - [*L*("no")]
    - `Locale.ROOT`

    If the locale is exactly `Locale("no", "NO", "NY")`, it is first converted to `Locale("nn", "NO")` and then the above procedure is followed.

    Also, Java treats the language "no" as a synonym of Norwegian Bokmål "nb". Except for the single case `Locale("no", "NO", "NY")` (handled above), when an input `Locale` has language "no" or "nb", candidate `Locale`s with language code "no" and "nb" are interleaved, first using the requested language, then using its synonym. For example, `Locale("nb", "NO", "POSIX")` generates the following candidate list:

    - [*L*("nb"), *C*("NO"), *V*("POSIX")]
    - [*L*("no"), *C*("NO"), *V*("POSIX")]
    - [*L*("nb"), *C*("NO")]
    - [*L*("no"), *C*("NO")]
    - [*L*("nb")]
    - [*L*("no")]
    - `Locale.ROOT`

    `Locale("no", "NO", "POSIX")` would generate the same list except that locales with "no" would appear before the corresponding locales with "nb".

The default implementation uses an `ArrayList` that overriding implementations may modify before returning it to the caller. However, a subclass must not modify it after it has been returned by `getCandidateLocales`.

For example, if the given `baseName` is "Messages" and the given `locale` is `Locale("ja", "", "XX")`, then a `List` of `Locale`s:

```
    Locale("ja", "", "XX")
    Locale("ja")
    Locale.ROOT
```

is returned. And if the resource bundles for the "ja" and "" `Locale`s are found, then the runtime resource lookup path (parent chain) is:

```
    Messages_ja -> Messages
```

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`locale` - the locale for which a resource bundle is desired

Returns:

a `List` of candidate `Locales` for the given `locale`

Throws:

`NullPointerException` - if `baseName` or `locale` is `null`

getFallbackLocale

```
public Locale getFallbackLocale(String baseName,
                                Locale locale)
```

Returns a `Locale` to be used as a fallback locale for further resource bundle searches by the `ResourceBundle.getBundle` factory method. This method is called from the factory method every time when no resulting resource bundle has been found for `baseName` and `locale`, where locale is either the parameter for `ResourceBundle.getBundle` or the previous fallback locale returned by this method.

The method returns `null` if no further fallback search is desired.

The default implementation returns the default `Locale` if the given `locale` isn't the default one. Otherwise, `null` is returned.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name for which `ResourceBundle.getBundle` has been unable to find any resource bundles (except for the base bundle)

`locale` - the `Locale` for which `ResourceBundle.getBundle` has been unable to find any resource bundles (except for the base bundle)

Returns:

a `Locale` for the fallback search, or `null` if no further fallback search is desired.

Throws:

`NullPointerException` - if `baseName` or `locale` is `null`

newBundle

```
public ResourceBundle newBundle(String baseName,
                                Locale locale,
                                String format,
                                ClassLoader loader,
                                boolean reload)
                         throws IllegalAccessException,
                                InstantiationException,
                                IOException
```

Instantiates a resource bundle for the given bundle name of the given format and locale, using the given class loader if necessary. This method returns `null` if there is no resource bundle available for the given parameters. If a resource bundle can't be instantiated due to an unexpected error, the error must be reported by throwing an `Error` or `Exception` rather than simply returning `null`.

If the `reload` flag is `true`, it indicates that this method is being called because the previously loaded resource bundle has expired.

Implementation Requirements:

Resource bundles in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. A resource bundle in a named module visible to the given class loader is accessible when the package of the resource file corresponding to the resource bundle is open unconditionally.

The default implementation instantiates a `ResourceBundle` as follows.

- The bundle name is obtained by calling `toBundleName(baseName, locale)`.
- If `format` is `"java.class"`, the `Class` specified by the bundle name is loaded with the given class loader. If the `Class` is found and accessible then the `ResourceBundle` is instantiated. The resource bundle is accessible if the package of the bundle class file is open unconditionally; otherwise, `IllegalAccessException` will be thrown. Note that the `reload` flag is ignored for loading class-based resource bundles in this default implementation.
- If `format` is `"java.properties"`, `toResourceName(bundlename, "properties")` is called to get the resource name. If `reload` is `true`, `load.getResource` is called to get a `URL` for creating a `URLConnection`. This `URLConnection` is used to disable the caches of the underlying resource loading layers, and to get an `InputStream`. Otherwise, `loader.getResourceAsStream` is called to get an `InputStream`. Then, a `PropertyResourceBundle` is constructed with the `InputStream`.

- If `format` is neither `"java.class"` nor `"java.properties"`, an `IllegalArgumentException` is thrown.
- If the `locale`'s language is one of the Legacy language codes, either old or new, then repeat the loading process if needed, with the bundle name with the other language. For example, "iw" for "he" and vice versa.

Parameters:

`baseName` - the base bundle name of the resource bundle, a fully qualified class name

`locale` - the locale for which the resource bundle should be instantiated

`format` - the resource bundle format to be loaded

`loader` - the `ClassLoader` to use to load the bundle

`reload` - the flag to indicate bundle reloading; `true` if reloading an expired resource bundle, `false` otherwise

Returns:

the resource bundle instance, or `null` if none could be found.

Throws:

`NullPointerException` - if `bundleName`, `locale`, `format`, or `loader` is `null`, or if `null` is returned by `toBundleName`

`IllegalArgumentException` - if `format` is unknown, or if the resource found for the given parameters contains malformed data.

`ClassCastException` - if the loaded class cannot be cast to `ResourceBundle`

`IllegalAccessException` - if the class or its nullary constructor is not accessible.

`InstantiationException` - if the instantiation of a class fails for some other reason.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

`IOException` - if an error occurred when reading resources using any I/O operations

See Also:

`ResourceBundleProvider.getBundle(String, Locale)`

getTimeToLive

```
public long getTimeToLive(String baseName,
                          Locale locale)
```

Returns the time-to-live (TTL) value for resource bundles that are loaded under this `ResourceBundle.Control`. Positive time-to-live values specify the number of milliseconds a bundle can remain in the cache without being validated against the source data from which it was constructed. The value 0 indicates that a bundle must be validated each time it is retrieved from the cache. `TTL_DONT_CACHE` specifies that loaded resource bundles are not put in the cache. `TTL_NO_EXPIRATION_CONTROL` specifies that loaded resource bundles are put in the cache with no expiration control.

The expiration affects only the bundle loading process by the `ResourceBundle.getBundle` factory method. That is, if the factory method finds a resource bundle in the cache that has expired, the factory method calls the `needsReload` method to determine whether the resource bundle needs to be reloaded. If `needsReload` returns `true`, the cached resource bundle instance is removed from the cache. Otherwise, the instance stays in the cache, updated with the new TTL value returned by this method.

All cached resource bundles are subject to removal from the cache due to memory constraints of the runtime environment. Returning a large positive value doesn't mean to lock loaded resource bundles in the cache.

The default implementation returns `TTL_NO_EXPIRATION_CONTROL`.

Parameters:

`baseName` - the base name of the resource bundle for which the expiration value is specified.

`locale` - the locale of the resource bundle for which the expiration value is specified.

Returns:

the time (0 or a positive millisecond offset from the cached time) to get loaded bundles expired in the cache, `TTL_NO_EXPIRATION_CONTROL` to disable the expiration control, or `TTL_DONT_CACHE` to disable caching.

Throws:

`NullPointerException` - if `baseName` or `locale` is `null`

needsReload

```
public boolean needsReload(String baseName,
                           Locale locale,
                           String format,
                           ClassLoader loader,
                           ResourceBundle bundle,
                           long loadTime)
```

Determines if the expired `bundle` in the cache needs to be reloaded based on the loading time given by `loadTime` or some other criteria. The method returns `true` if reloading is required; `false` otherwise. `loadTime` is a millisecond offset since the `Calendar` Epoch.

The calling `ResourceBundle.getBundle` factory method calls this method on the `ResourceBundle.Control` instance used for its current invocation, not on the
instance used in the invocation that originally loaded the resource bundle.

The default implementation compares `loadTime` and the last modified time of the source data of the resource bundle. If it's determined that the source data has been modified since `loadTime`, `true` is returned. Otherwise, `false` is returned. This implementation assumes that the given `format` is the same string as its file suffix if it's not one of the default formats, `"java.class"` or `"java.properties"`.

Parameters:

`baseName` - the base bundle name of the resource bundle, a fully qualified class name

`locale` - the locale for which the resource bundle should be instantiated

`format` - the resource bundle format to be loaded

`loader` - the `ClassLoader` to use to load the bundle

`bundle` - the resource bundle instance that has been expired in the cache

`loadTime` - the time when `bundle` was loaded and put in the cache

Returns:

`true` if the expired bundle needs to be reloaded; `false` otherwise.

Throws:

`NullPointerException` - if `baseName`, `locale`, `format`, `loader`, or `bundle` is `null`

toBundleName

```
public String toBundleName(String baseName,
                           Locale locale)
```

Converts the given `baseName` and `locale` to the bundle name. This method is called from the default implementation of the `newBundle` and `needsReload` methods.

This implementation returns the following value:

```
    baseName + "_" + language + "_" + script + "_" + country + "_" + variant
```

where `language`, `script`, `country`, and `variant` are the language, script, country, and variant values of `locale`, respectively. Final component values that are empty Strings are omitted along with the preceding '_'. When the script is empty, the script value is omitted along with the preceding '_'. If all of the values are empty strings, then `baseName` is returned.

For example, if `baseName` is `"baseName"` and `locale` is `Locale("ja", "", "XX")`, then `"baseName_ja_ _XX"` is returned. If the given locale is `Locale("en")`, then `"baseName_en"` is returned.

Overriding this method allows applications to use different conventions in the organization and packaging of localized resources.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`locale` - the locale for which a resource bundle should be loaded

Returns:

the bundle name for the resource bundle

Throws:

`NullPointerException` - if `baseName` or `locale` is `null`

See Also:

`AbstractResourceBundleProvider.toBundleName(String, Locale)`

toResourceName

```
public final String toResourceName(String bundleName,
                                   String suffix)
```

Converts the given `bundleName` to the form required by the `ClassLoader.getResource` method by replacing all occurrences of `'.'` in `bundleName` with `'/'` and appending a `'.'` and the given file `suffix`. For example, if `bundleName` is `"foo.bar.MyResources_ja_JP"` and `suffix` is `"properties"`, then `"foo/bar/MyResources_ja_JP.properties"` is returned.

Parameters:

`bundleName` - the bundle name

`suffix` - the file type suffix

Returns:

the converted resource name

Throws:

`NullPointerException` - if `bundleName` or `suffix` is `null`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.